

Notification Service Design Document

Purpose: Deliver Email, SMS and Push notifications asynchronously using Kafka. The Notification Service is the source of truth for notification delivery history and is responsible for retrying failed notifications.

Responsibilities

- Send Email notifications
- Send SMS notifications
- Send Push notifications
- Retry failed notifications
- Maintain notification delivery history
- Publish NotificationSent event

Owned Entity

NotificationLog

Source of Truth

Notification delivery history and notification status.

High-Level Architecture

BookingConfirmed BookingCancelled PaymentFailed | v Kafka | v
Notification Service | +-----> Email Provider | +-----> SMS Provider |
+-----> Push Provider | v NotificationLog | v NotificationSent

Database Schema (notification_db)

NotificationLog ----- notification_id (PK)
booking_id notification_type EMAIL SMS PUSH recipient status PENDING SENT
FAILED provider_reference retry_count error_message created_at sent_at

Suggested Indexes

NotificationLog(booking_id)
NotificationLog(status)
NotificationLog(created_at)
NotificationLog(provider_reference)

Kafka Events Consumed

BookingConfirmed
BookingCancelled
PaymentFailed

Kafka Events Published

NotificationSent

Notification Workflow

BookingConfirmed → Create NotificationLog → Send Email/SMS/Push → Update
Notification Status → Publish NotificationSent

Failure Handling

If notification delivery fails:

- Update NotificationLog with FAILED status
- Increment retry_count
- Store error_message
- Retry using exponential backoff
- Move permanently failed messages to monitoring/dashboard

Business Rules

- Notification processing must be idempotent.
- NotificationLog stores only metadata, not the complete message body.
- One event may generate multiple notifications (Email + SMS + Push).
- NotificationSent is published only after successful delivery.

Scalability & Reliability

- Stateless Notification Service instances
- Kafka consumer groups for horizontal scaling
- Idempotent event processing
- Retry with exponential backoff
- Structured logging
- Metrics using Prometheus
- Distributed tracing